of 5, price less than a certain amount, and so on. The UI may support various widgets such as dropdowns, sliders, check boxes, and so forth to enable filtering the search even further.

This chapter introduces search and the fundamental features we can use while searching. We first discuss search mechanics: how a search request is processed and a response is created and sent to the client. We then look at fundamentals of search: the search API and contexts in which search queries are performed. We dissect a request and a response to dig deeper into their components.

We also look at types of searches, the URI request, and Query DSL, as well as why we prefer Query DSL over the URI request method. Finally, we examine crosscutting search features such as highlighting, pagination, explanation, and manipulating response fields.

> **NOTE** The code for this chapter is available on GitHub (http://mng.bz/e1yZ) and on the book's website (https://www.manning.com/books/elasticsearch-in-action-second-edition).

8.1 Overview

Elasticsearch supports simple search functionality and advanced searches that consider multiple criteria, including geospatial queries. Broadly speaking, Elasticsearch deals with two variants of search: *structured search* and *unstructured search*. We've gone over these variants in earlier chapters, so this is pretty much a recap; feel free to skip this section.

Structured search returns results with no associated *relevance* score. Elasticsearch fetches documents if they match exactly and doesn't worry about whether they are a close match or how well they match. Searches for flights between a set of dates, best-selling books during a particular sales promotion, and so on fall into this category. When the search is carried out, Elasticsearch checks whether the match is successful: for example, there are or are not flights that fall in the given date range, and there are or are not best-selling books. Nothing falls into a *maybe* category. This type of structured search is provided by *term-level queries* in Elasticsearch.

On the other hand, in an unstructured search, Elasticsearch retrieves results that are closely related to the query. Results are scored based on how relevant they are to the criteria: highly relevant results get higher scores and so are positioned at the top of the list. Searching on `text` fields yields relevant results, and Elasticsearch provides full-text search to search unstructured data.

We communicate with the Elasticsearch engine using a RESTful API to execute queries. A search query is written using a special query syntax called Query DSL (domain-specific language) or a URL standard called *query requests*. When a query is issued, any available node in the cluster picks the request and processes it. The response is returned as a JSON object, with an array of individual documents as results in the object.